UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

Proyecto Final — Un sistema distribuido para una pregunta del caso

Docente: Heyder Medrano

Modalidad: En grupos de 3. Cada grupo trabaja una pregunta de negocio del caso de viajes de Uber y Lyft (TLC, junio de 2025, 19.868.009 viajes).

Fechas: Arranca el sábado 31 de octubre a las 11:00 a. m. (elección de la pregunta y plan). Se sustenta el sábado 7 de noviembre entre las 2:15 y las 3:15 p. m.

Entorno: El codespace de cada estudiante (Preparación Previa del Taller 1). Los programas del proyecto ya vienen en tpd/hadoop y tpd/spark.

Entregables: Informe técnico (3 a 5 páginas), tablero con la respuesta a la gerencia y sustentación de 10 minutos.

Términos: si un término no te resulta claro, búscalo en Glosario_Tecnico_Tecnologias_Procesamiento_Datos.docx (carpeta Sesion_1_Vie_30_Octubre/Estudiantes).

1. El reto

La gerencia de operaciones tiene ocho preguntas sobre el mes de junio de 2025. Cada grupo toma una y entrega la respuesta completa y exacta, procesando los 19,9 millones de viajes con un sistema distribuido que configura, mide y justifica. No basta con el número: hay que demostrar con evidencia qué arquitectura, qué motor y qué formato de datos conviene para esa pregunta, y qué cambia si los datos crecen a un año.

Es la continuación de los talleres del sábado: el cálculo por empresa del Taller 1A ya resolvió una pregunta; ahora cada grupo resuelve otra, con las mismas herramientas y la misma forma de medir.

2. Las ocho preguntas

Se eligen el sábado 31 a las 11:00; dos grupos no pueden tener la misma pregunta. Las columnas son las que entrega el programa de esa pregunta.

#	Pregunta de la gerencia	Lo que entrega el programa
p1	¿A qué horas hay más viajes y qué parte de la tarifa se queda la plataforma en cada hora?	Por hora: viajes, tarifa base, parte de la plataforma (US$ y %). 24 filas.
p2	¿Cómo cambian los viajes y la tarifa promedio según el día de la semana y la empresa?	Por día y empresa: viajes, tarifa base y tarifa promedio. 14 filas.
p3	¿Dónde y con qué empresa se dejan más propinas?	Por borough de origen y empresa: viajes, viajes con propina (y %), propina total y promedio cuando hay. 12 filas.
p4	¿Cuánto mueven los aeropuertos (Newark, JFK y LaGuardia), desde y hacia ellos?	Por aeropuerto y sentido: viajes, tarifa promedio, cargo de aeropuerto y parte de la plataforma.
p5	¿Cuánto se le paga al conductor por milla, según el borough y la empresa?	Por borough y empresa: viajes, pago al conductor, millas y pago por milla. 12 filas.
p6	¿A qué horas es más lento moverse por la ciudad?	Por hora: viajes, millas, horas de viaje, velocidad promedio (mph) y duración promedio. 24 filas.
p7	¿Qué zonas de recogida le dejan más dinero a la plataforma?	Por zona: nombre y borough, viajes, tarifa base, parte de la plataforma total y por viaje. 261 filas.

#	Pregunta de la gerencia	Lo que entrega el programa
p8	¿Cuánto recauda el nuevo cargo de congestión de Manhattan y a qué horas?	Por hora: viajes, viajes con cargo (y %) y recaudo. 24 filas.

⚠ Ojo con los promedios: junio de 2025 tuvo 5 lunes y 5 domingos, y 4 de cada uno de los demás días. Comparar totales por día sin tenerlo en cuenta lleva a una conclusión equivocada (p2).

3. Los programas: ya están en tu codespace

La misma pregunta viene resuelta para dos motores. El grupo los ejecuta, los compara, los ajusta y justifica cuál usaría:

- `tpd/hadoop/mapper_proyecto.py` y `reducer_proyecto.py`: Hadoop Streaming. El mapper emite enteros (centavos, centésimas de milla, segundos) para que la suma sea exacta y el reducer sirva también como combiner.
- `tpd/spark/job_proyecto.py`: Spark con DataFrames; lee el CSV o, con --parquet, el Parquet oficial.

⚠ Si tu codespace se creó con la preparación anterior (zip y repositorio propio), no tiene estos programas. Tráelos con el comando del recuadro siguiente.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
for f in hadoop/mapper_proyecto.py hadoop/reducer_proyecto.py spark/job_proyecto.py; \
    do curl -sSfo $f \
    https://raw.githubusercontent.com/hmedrano-gif/tpd-codespace/main/tpd/$f; done; ls \
    -l hadoop/*_proyecto.py spark/job_proyecto.py
```

Con Hadoop (ejemplo con la pregunta p3)

Con Hadoop encendido (bash encender_hadoop.sh) y el mes cargado en HDFS como en el Taller 1A, dentro del contenedor namenode:

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar -files \
    /opt/codigo/mapper_proyecto.py,/opt/codigo/reducer_proyecto.py,/opt/codigo/zonas.csv \
    -mapper "python3 mapper_proyecto.py p3" -combiner \
    "python3 reducer_proyecto.py p3 --combinar" -reducer \
    "python3 reducer_proyecto.py p3" -input /caso/input/fhvhv_2025-06.csv -output /caso/p3
```

Comando 2 de 2

```
hdfs dfs -cat /caso/p3/part-*
```

Cambia `p3` por tu pregunta en las cuatro partes del comando. La primera línea del resultado, que empieza con #, son los nombres de las columnas. En la prueba, la pregunta p3 tardó **6 minutos y 8 segundos** con 4.096 MB para YARN.

Con Spark (CSV y Parquet)

Con Hadoop apagado (docker compose stop), en la terminal del codespace:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_proyecto.py --pregunta p3
```

Comando 2 de 2

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_proyecto.py --pregunta p3 \
    --parquet
```

El resultado queda en `tpd/spark/output_proyecto_p3` (un CSV con encabezado) y se imprime en pantalla. Al final el programa espera Enter para que alcances a revisar la Spark UI (puerto 4040): presiónalo para cerrar Spark. En la prueba, con el Parquet y 4 núcleos, las ocho preguntas tardaron entre 15 y 35 s cada una, y sus cifras coinciden al centavo con las de Hadoop.

Sacar el resultado para el tablero

- De Hadoop: dentro del namenode, `hdfs dfs -getmerge /caso/p3 /opt/codigo/p3_hadoop.tsv` junta el resultado en un solo archivo y lo deja en tpd/hadoop (en la prueba: el encabezado con # y las 12 filas).
- De Spark: el resultado ya está en `tpd/spark/output_proyecto_pN`, en el archivo part-….csv.
- Ábrelo con un clic en el explorador del codespace (panel izquierdo): se ve como una tabla. Selecciona todo con Ctrl + A, cópialo con Ctrl + C y pégalo en una hoja de cálculo (Google Sheets o Excel), que es la fuente del tablero.

4. Lo que entrega el grupo

Entregable	Qué debe tener
1. Diseño	Un diagrama de la arquitectura: de dónde salen los datos (CSV o Parquet), dónde se guardan (HDFS o disco), qué motor los procesa, dónde queda el resultado y cómo llega al tablero. Por qué ese motor y ese formato para su pregunta.
2. Configuración y medición	La pregunta resuelta con Hadoop y con Spark, con el mismo resultado al centavo. Al menos dos ajustes medidos (por ejemplo: combiner, memoria de YARN, núcleos de Spark, CSV frente a Parquet), con una tabla de tiempos, datos leídos y shuffle tomada de las consolas (historial 19888 y Spark UI).
3. Integración de datos	El resultado unido con la tabla oficial de zonas cuando aplica (nombres y boroughs), verificado (Hadoop = Spark) y cargado en un tablero de Looker Studio o Power BI.
4. Tablero	Dos o tres visuales que respondan la pregunta de la gerencia, con la cifra clave a la vista y una conclusión en una frase.
5. Informe técnico	De 3 a 5 páginas: la pregunta y su respuesta; la arquitectura; la tabla de mediciones; qué recurso fue el cuello de botella; qué cambiaría con un año de datos (12 veces más) y en la nube.
6. Sustentación	10 minutos por grupo el sábado 7 de noviembre (2:15–3:15 p. m.): los tres integrantes hablan; se muestra el tablero y una medición en vivo o su evidencia.

5. Cómo se evalúa

Los tres criterios del contenido programático para el Proyecto Final, con el mismo peso:

Criterio	Excelente	Aceptable	Insuficiente
Configuración del sistema	Ejecuta con Hadoop y con Spark, ajusta al menos dos parámetros y explica con evidencia qué cambió y por qué.	Ejecuta con los dos motores, pero los ajustes no están medidos o no se explican.	Solo un motor, o resultados sin evidencia de la ejecución.
Integración de datos	Resultado completo y exacto (Hadoop = Spark al centavo), unido con las zonas y llevado al tablero sin pérdidas.	Resultado correcto, pero sin verificación cruzada o con pasos manuales sin explicar.	Cifras que no coinciden o datos incompletos.
Rendimiento	Tabla de mediciones con tiempos, datos leídos y shuffle; identifica el cuello de botella y propone la arquitectura para un año de datos.	Mide tiempos, pero no identifica el cuello de botella o no lo justifica.	No hay mediciones o se reportan sin fuente.

6. Calendario

Cuándo	Qué
Sábado 31 de octubre, 11:00–11:30	Cada grupo elige su pregunta y escribe su plan: qué va a medir y qué ajustes va a probar.
Del 1 al 6 de noviembre	Ejecución y mediciones en el codespace; tablero. Cuiden las horas: cada hora encendido gasta 4 de las 120 gratis del mes; apaguen el codespace al terminar.
Viernes 6 de noviembre	Dudas en clase.
Sábado 7 de noviembre, 2:15–3:15	Sustentación y entrega del informe y del tablero.

✓ **Punto de control:** antes de sustentar, el grupo comprueba que su resultado de Hadoop y el de Spark son idénticos al centavo.